

Stocker un fichier sur différentes machines

Ce qu'on a vu la dernière fois

Une définition

Le Big Data commence là où les outils classiques ne suffisent plus.

Les 5 V

Volume, Vélocité, Variété, Véracité, Valeur : la grille de lecture du domaine.

Le scale-out

La réponse technique : de nombreuses machines ordinaires qui coopèrent.

On avait conclu qu'il fallait répartir la donnée sur beaucoup de machines. Aujourd'hui : **avec quel outil ?** Réponse : Apache Hadoop, et son système de stockage, HDFS.

1 To

Comment stocker et lire un fichier trop gros pour n'importe quelle machine, de façon fiable, même si des machines tombent en panne ?

La réponse tient en un mot : **Hadoop** . On découvre ce qu'il est, puis comment sa couche de stockage, HDFS, résout ce problème.

À la fin de la séance, vous savez

Expliquer ce qu'est Apache Hadoop et citer ses 4 modules.

Décrire les modes de déploiement : single node, cluster.

Expliquer l'architecture de HDFS : NameNode, DataNode, blocs, réplication.

Calculer le nombre de blocs et l'espace occupé par un fichier.

Manipuler HDFS en ligne de commande et observer la tolérance aux pannes.

00

Apache Hadoop, le cadre

Qu'est-ce qu'Apache Hadoop ?

Un **framework de calcul distribué** conçu pour passer d'un serveur unique à des milliers de machines, chacune offrant du calcul et du stockage **locaux**.

Il **détecte et gère les pannes au niveau de la couche application** : le service reste hautement disponible sur un cluster dont chaque machine peut tomber.

L'idée forte : assumer que les machines tombent et régler cela **dans le logiciel** . C'est ce qui permet d'utiliser du matériel bon marché, en grand nombre.



Les quatre modules de Hadoop

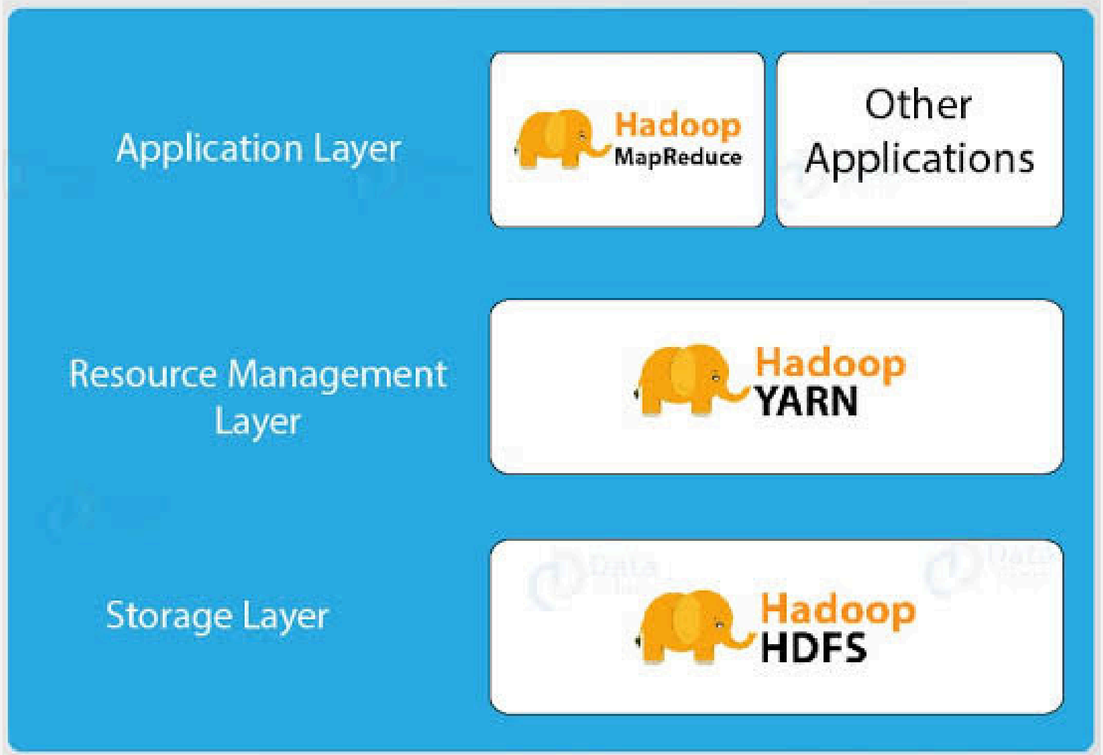
MODULE	ROLE	QUAND
Hadoop Common	Les utilitaires de base qui soutiennent les autres modules	Transverse
Hadoop HDFS	Le système de fichiers distribué : stockage à haut débit	**Cette séance**
Hadoop YARN	Le gestionnaire de ressources : planification, partage du cluster	Séance 5
Hadoop MapReduce	Le moteur de traitement parallèle, basé sur YARN	Séances 3 et 4

Voyez Hadoop comme le système d'exploitation du cluster : HDFS est son disque dur, YARN son ordonnanceur, MapReduce son moteur de calcul.

Comment les modules s'empilent

De bas en haut : **HDFS** stocke, **YARN** distribue les ressources, **MapReduce** calcule. **Hadoop Common** traverse les trois couches.

Chaque couche ne parle qu'à sa voisine : on peut remplacer le moteur de calcul sans toucher au stockage.



Qui l'utilise, et quelle version ?

Adoption

Une grande variété d'entreprises l'utilisent, en recherche comme en production : Amazon, Adobe, Alibaba, eBay, Facebook, Blablacar, et beaucoup d'autres. La liste complète est tenue dans le wiki « PoweredBy » d'Apache Hadoop.

Versions et téléchargement

Plusieurs versions majeures coexistent : les distributions **3.3.x** et **3.4.x** sont les plus courantes aujourd'hui. Téléchargement officiel sur hadoop.apache.org/releases.html .

Attention aux tutoriels en ligne : beaucoup datent de Hadoop 2 et donnent des commandes et des ports qui ne fonctionnent plus.

Prérequis et modes de déploiement

Prérequis

Java 11, requis. **SSH** , car les démons se lancent via SSH, même en local.

Single node

Mode pseudo-distribué : tout sur une machine, chaque démon dans un processus Java séparé. Pour apprendre et tester, c'est notre cas en TP.

Cluster

Plusieurs machines, un nœud maître et des nœuds esclaves. Pour la production. Hadoop a été testé jusqu'à des clusters de **2000 nœuds**.

Les grandes étapes d'installation, détaillées dans le guide de prérequis : créer un utilisateur dédié, importer la clé, télécharger et décompresser Hadoop, définir JAVA_HOME et HADOOP_HOME.

PARTIE 1

01

HDFS, le stockage

Classique ou distribué

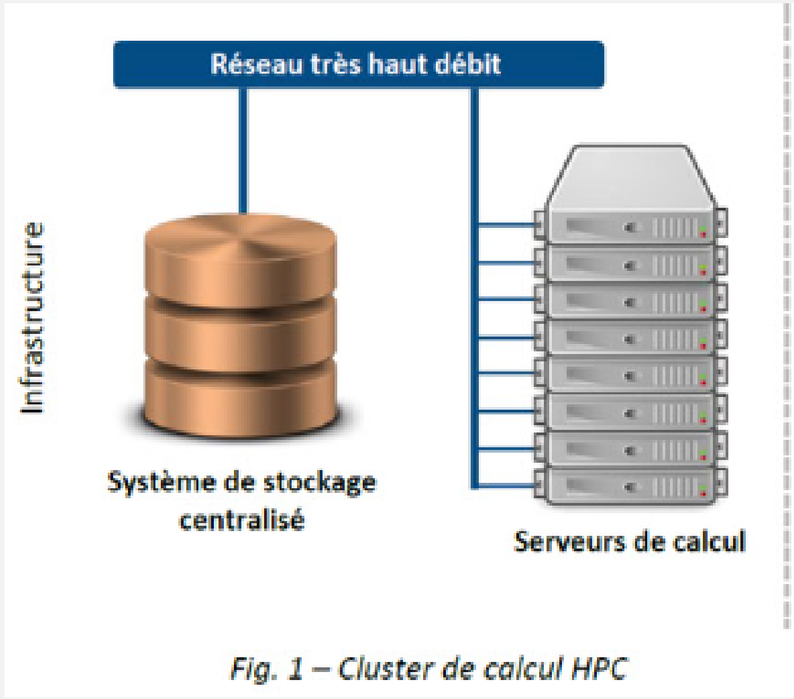


Fig. 1 – Cluster de calcul HPC

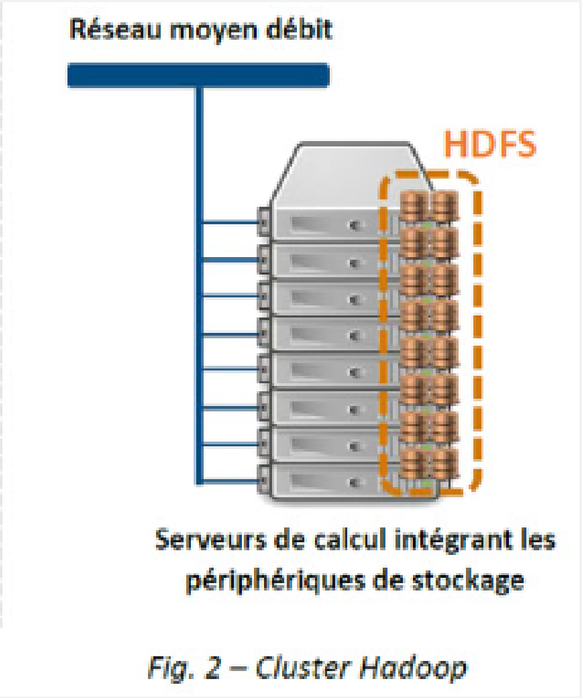


Fig. 2 – Cluster Hadoop

Classique : les fichiers vivent sur un seul disque.

Distribué : les fichiers sont répartis sur plusieurs machines reliées en réseau.

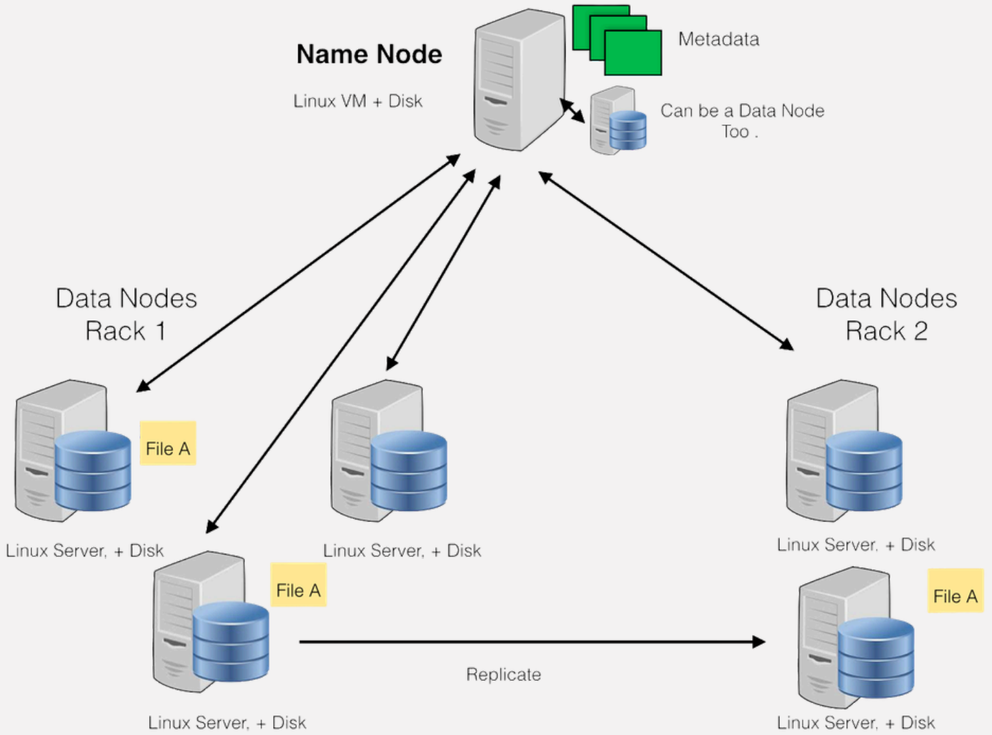
L'utilisateur, lui, voit un chemin unique : /donnees/fichier.txt.

Idée clé : la distribution est **transparente** pour l'utilisateur et les applications.

HDFS en une phrase

Hadoop Distributed File System : le module de stockage de Hadoop, un système de fichiers distribué qui s'exécute sur du **matériel standard** et donne un accès **à haut débit** aux données, de façon transparente.

Ce qui le rend particulier : il est conçu pour de **très gros fichiers** , un **débit élevé** en lecture et une **tolérance aux pannes** native, sur du matériel bon marché qui tombe souvent.



Les quatre fondements

Les pannes sont la norme

Avec des milliers de machines ordinaires, il y en a toujours une en panne. HDFS est conçu pour ça, pas contre ça.

Le débit prime sur la latence

Lire en continu de gros volumes compte plus qu'accéder vite à un petit morceau précis.

Gros fichiers

Optimisé pour des fichiers de plusieurs Go ou To, pas pour des millions de petits fichiers.

Write once, read many

Un fichier est écrit une fois, puis lu souvent, et n'est pas modifié. La cohérence en devient simple.

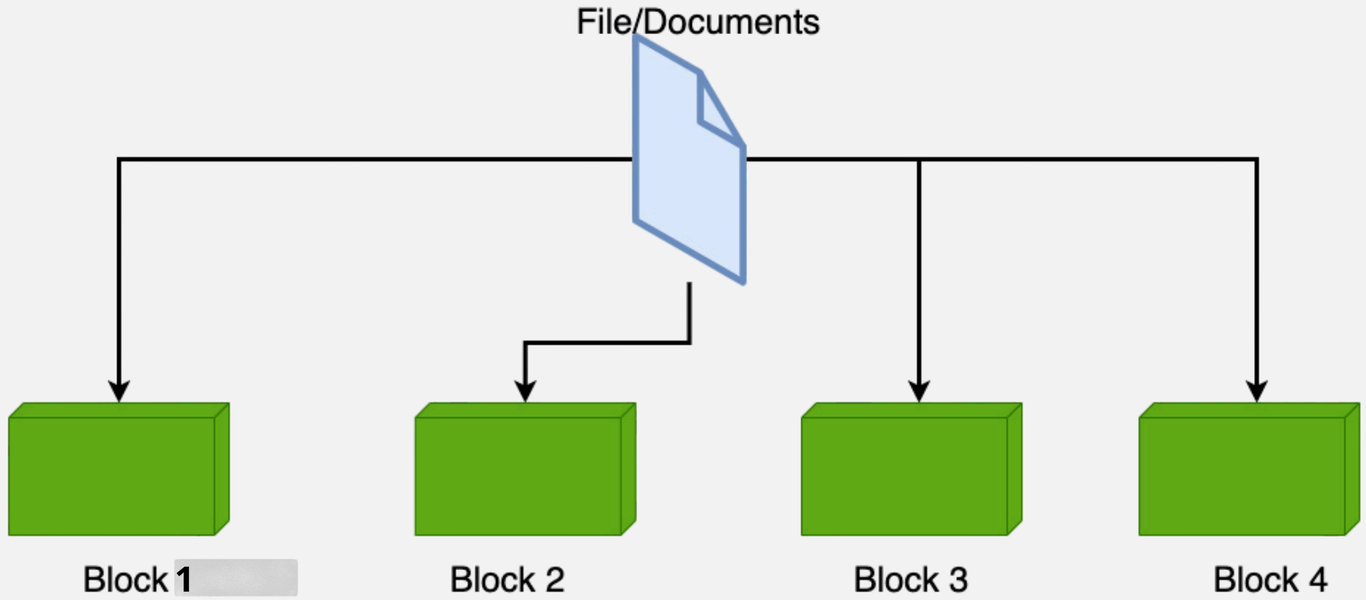
Le découpage en blocs

HDFS découpe chaque fichier en blocs de taille fixe, **128 Mo** par défaut.

Chaque bloc est stocké sur une machine différente du cluster.

On dépasse ainsi la limite d'un seul disque : il suffit d'ajouter des machines.

On peut lire **plusieurs blocs en parallèle** , donc à débit élevé.



4

blocs pour un fichier de 512 Mo : 4 × 128 Mo,
répartis sur 4 machines.

Le dernier bloc n'occupe que ce qu'il contient : HDFS ne réserve pas 128 Mo pour 10 Mo de données.

La réplication, facteur 3

Le principe

Chaque bloc est copié sur trois machines par défaut. Si l'une tombe, le bloc existe toujours ailleurs.

Le compromis

1 Go de données occupe 3 Go sur le cluster. C'est le prix de la fiabilité, accepté car le matériel est bon marché.

L'auto-réparation

Si une copie disparaît, HDFS en recrée une ailleurs pour revenir au facteur de réplication.

Retenir la règle de calcul : **espace réel = taille du fichier × facteur de réplication**.

PARTIE 2

02

L'architecture maître / esclaves

NameNode et DataNodes

NameNode, le maître

Stocke toutes les **métadonnées** : noms, emplacement des blocs, attributs, droits.

Les garde **en RAM** pour une recherche immédiate.

La taille du système de fichiers est donc limitée par sa RAM.

Il ne stocke **aucune donnée** de fichier, seulement la carte.

DataNodes, les ouvriers

Stockent le **contenu réel** des fichiers, sous forme de blocs.

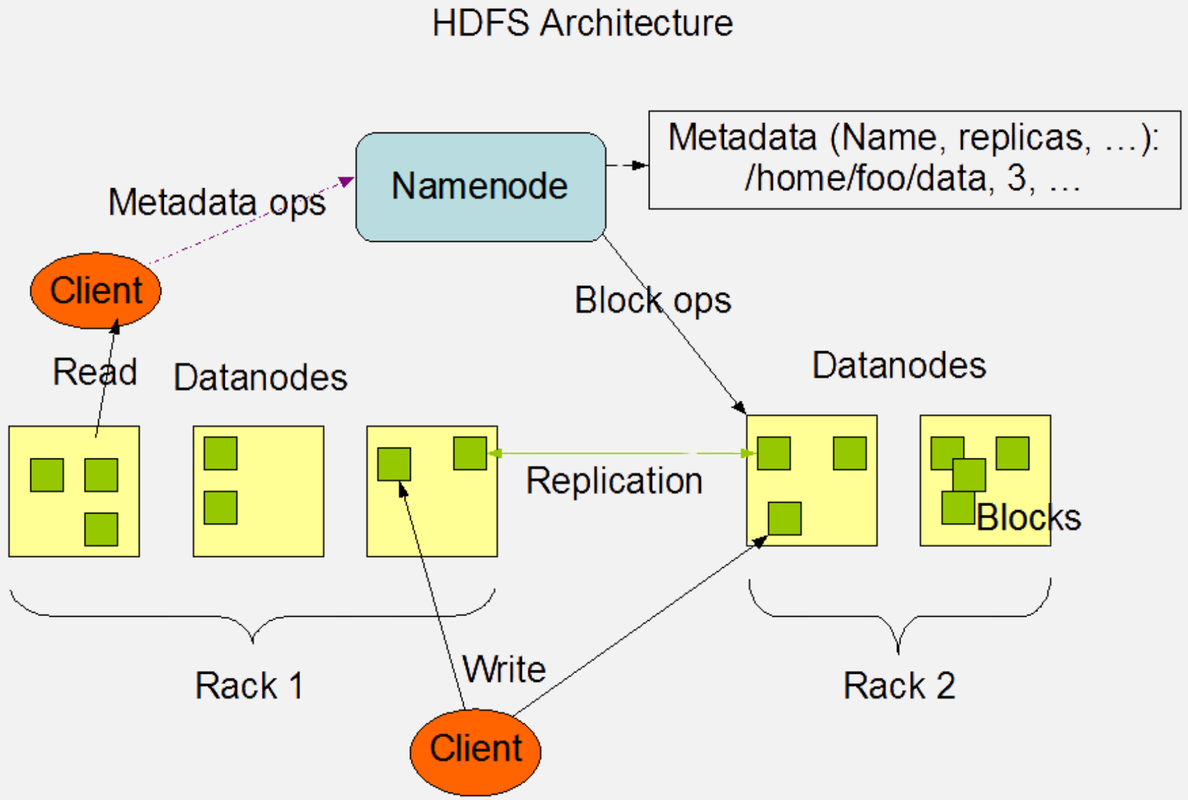
Les blocs d'un même fichier vivent sur des DataNodes différents.

Un même bloc est répliqué sur plusieurs DataNodes.

Ils envoient périodiquement au NameNode un rapport de leurs blocs.

Le cluster en une figure

Le client interroge le NameNode pour obtenir la carte des blocs, puis échange les données **directement** avec les DataNodes. Les blocs se répliquent entre les nœuds, y compris entre deux baies.



« Le NameNode est le bibliothécaire : il sait où est chaque livre mais ne le porte pas. Les DataNodes sont les étagères. »

Sans le bibliothécaire, on a les livres mais on ne sait plus où ils sont.

EXERCICE 1 · BINÔME

Calculer blocs, copies et espace

Cluster HDFS, taille de bloc **128 Mo**, facteur de réplication **3**. On stocke un fichier de **500 Mo**.

Combien de blocs seront créés ? Quelle est la taille du dernier ?
Combien de copies de blocs au total sur le cluster ?
Quel espace disque réel ce fichier occupe-t-il ?
Le NameNode stocke-t-il une partie du contenu ? Justifiez.

PARTIE 3

03

La tolérance aux pannes

Quand un DataNode tombe

Le battement de cœur

Chaque DataNode envoie régulièrement un heartbeat au NameNode pour signaler qu'il est vivant.

La détection

Sans heartbeat pendant un certain temps, le nœud est considéré comme perdu.

La réparation

Le NameNode retrouve les autres copies des blocs perdus et les réplique ailleurs.

Le tout **sans intervention humaine** et sans interruption de service : c'est l'auto-réparation du cluster.

Le point critique : la panne du NameNode

Perdre un DataNode n'est pas grave : les copies existent ailleurs.

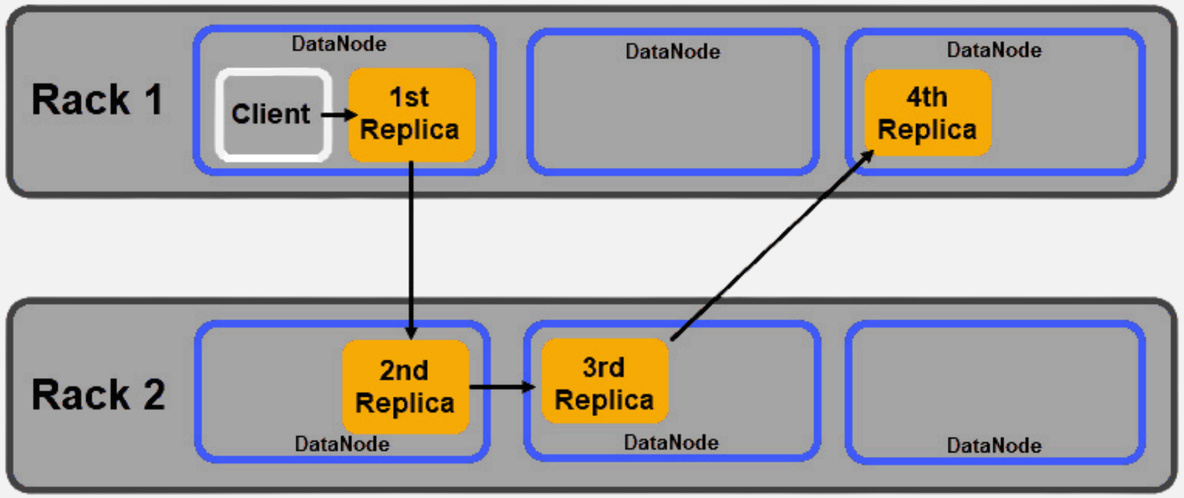
Perdre le NameNode, c'est perdre la **carte** de tous les fichiers : on garde les blocs sans savoir lesquels forment quel fichier.

Parade 1 : **sauvegarder les métadonnées** sur un stockage sûr, montage local et NFS.

Parade 2 : un **NameNode secondaire** ou une configuration haute disponibilité à deux NameNodes.

À retenir : la donnée est protégée par la réplication, les **métadonnées** doivent l'être autrement.

Le placement par rack



1 copie sur le **nœud local**, là où écrit le client.

1 copie sur un nœud d'un **autre rack**.

1 copie sur un **troisième nœud du rack distant**.

Si un rack entier tombe (pane d’alimentation, de switch), une copie survit ailleurs.

Le client lit toujours depuis le nœud **le plus proche**, pour minimiser le trafic réseau.

Écrire et lire un fichier

L'écriture

Le client demande au NameNode où écrire ; il reçoit une liste de DataNodes.

Il écrit le premier bloc sur le premier DataNode.

Ce DataNode transmet la copie au suivant (**pipeline de réplication**), et ainsi de suite.

Une fois toutes les copies écrites, on passe au bloc suivant.

La lecture

Le client demande au NameNode l'emplacement des blocs.

Le NameNode renvoie, pour chaque bloc, le meilleur DataNode.

Le client contacte **directement** les DataNodes pour récupérer les blocs

Le NameNode ne fait que guider : il ne transporte jamais les données.

Comme les données ne passent pas par le NameNode, celui-ci ne devient pas un goulot d'étranglement, même avec des milliers de clients.

Les commandes essentielles

Fichiers et dossiers

`hdfs dfs -mkdir /user/etudiant`	Créer un répertoire
`hdfs dfs -put fichier.txt /user/etudiant/`	Local vers HDFS
`hdfs dfs -get /user/etudiant/fichier.txt .`	HDFS vers local
`hdfs dfs -rm /user/etudiant/fichier.txt`	Supprimer un fichier

Inspection

`hdfs dfs -ls /user/etudiant`	Lister un répertoire
`hdfs dfs -cat /user/etudiant/fichier.txt`	Afficher le contenu
`hdfs dfs -df -h`	Espace total et libre
`hdfs fsck /chemin -files -blocks -locations`	Blocs et emplacements

Toutes les commandes reprennent la logique Linux.

PARTIE 4

04

La pratique en ligne de commande

TP guidé : démarrer et utiliser le cluster

La réponse à la question du début

Hadoop est le framework de calcul distribué, **HDFS** en est le stockage.

Découper le fichier en blocs de 128 Mo répartis, pour dépasser la limite d'un disque.

Répliquer chaque bloc trois fois, pour survivre aux pannes.

Séparer les rôles : le NameNode garde la carte, les DataNodes portent les blocs.

Se réparer seul grâce aux heartbeats et à la re-réplication automatique.

Trois idées à emporter

Hadoop = un framework distribué en 4 modules : Common, HDFS, YARN, MapReduce.

HDFS découpe les gros fichiers en **blocs** répartis et les **réplique** ; NameNode = la carte, DataNodes = les données.

La **tolérance aux pannes** est native : heartbeats, re-réplication, protection par rack.

Question qui nous attend : maintenant qu'on sait stocker, comment **traiter** ces données en parallèle sur toutes les machines ? Réponse : MapReduce en séance 3, appuyé sur YARN.

EXERCICE 2 · PARTIE A · INDIVIDUEL

Vrai ou faux, justifié

HDFS, YARN et MapReduce sont trois modules de Hadoop.

Le NameNode stocke le contenu des fichiers.

Par défaut, chaque bloc est répliqué 3 fois.

Si un DataNode tombe, ses données sont définitivement perdues.

Le client lit les blocs directement depuis les DataNodes, pas via le NameNode.

MapReduce

Traiter en parallèle ce que HDFS a su stocker.